HDOC Day-16

Question: Menu-Driven Program for Digit-Based Operations Using User-Defined Functions

1. Problem Statement & Identification

- **Problem:** Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer using separate user-defined functions (void return type and no arguments) for each option:
    1. Check whether digits are in increasing order from left to right.
    2. Check whether digits are in decreasing order from left to right.
    3. Find the largest digit and the smallest digit in the number.
    4. Find the second largest digit and second smallest digit in the number.
- **Input:** Menu choice (choice), and inside each function, a positive integer (num).
- **Output:** Display result directly inside the corresponding user-defined function.

2. Algorithm

1. **Start**

2. Display menu options (1 to 4).
3. Read user choice.
4. Execute user-defined function based on choice using switch-case:
    - **Case 1: checkIncreasing()**
        1. Read num. Set temp = num, prev_digit = 10, is_inc = 1.
        2. Extract digits from right to left (current_digit = temp % 10).
        3. If current_digit >= prev_digit, set is_inc = 0 and break.
        4. Update prev_digit = current_digit and temp /= 10.
        5. Display whether digits are in increasing order.
    - **Case 2: checkDecreasing()**
        1. Read num. Set temp = num, prev_digit = -1, is_dec = 1.
        2. Extract digits from right to left (current_digit = temp % 10).
        3. If current_digit <= prev_digit, set is_dec = 0 and break.

4. Update prev_digit = current_digit and temp /= 10.
5. Display whether digits are in decreasing order.

- **Case 3: findLargestSmallest()**
  1. Read num. Set largest = -1, smallest = 10.
  2. Extract digits, update largest if digit > largest, and smallest if digit < smallest.
  3. Display largest and smallest digits.
- **Case 4: findSecondLargestSmallest()**
  1. Read num. Populate digit frequency array freq[10].
  2. Traverse freq from 0 to 9 to find smallest and second_smallest.
  3. Traverse freq from 9 to 0 to find largest and second_largest.
  4. Display second_largest and second_smallest digits.
- **Default:** Print "Invalid Choice!".

5. **End**

3. Test Case Table

Test Case	Choice	Input Number	Operation	Expected Output	Actual Output	Status
1	1	12345	Check Increasing Order	Digits are in increasing order	Digits are in increasing order	Pass
2	2	54321	Check Decreasing Order	Digits are in decreasing order	Digits are in decreasing order	Pass
3	3	38195	Largest & Smallest	Largest = 9, Smallest = 1	Largest = 9, Smallest = 1	Pass
4	4	38195	Second Largest &	Second Largest = 8, Secon	Second Largest = 8, Secon	Pass

Test Case	Choice	Input Number	Operation	Expected Output	Actual Output	Status
			Smallest	d Smallest = 3	d Smallest = 3	

4. C Source Code

```
#include <stdio.h>

// Function Prototypes (void return type, no parameters)
void checkIncreasing(void);
void checkDecreasing(void);
void findLargestSmallest(void);
void findSecondLargestSmallest(void);

int main(void) {
   int choice;

   printf("--- MENU ---\n");
   printf("1. Check whether digits are in increasing order\n");
   printf("2. Check whether digits are in decreasing order\n");
   printf("3. Find the largest and smallest digit\n");
   printf("4. Find the second largest and second smallest digit\n");
   printf("Enter your choice (1-4): ");
   scanf("%d", &choice);

   switch (choice) {
      case 1:
         checkIncreasing();
         break;
      case 2:
         checkDecreasing();
```

```
            break;
        case 3:
            findLargestSmallest();
            break;
        case 4:
            findSecondLargestSmallest();
            break;
        default:
            printf("\nInvalid Choice!\n");
            break;
    }

    return 0;
}

void checkIncreasing(void) {
    int num, temp, prev_digit = 10, is_inc = 1;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;
    while (temp > 0) {
        int current_digit = temp % 10;
        if (current_digit >= prev_digit) {
            is_inc = 0;
            break;
        }
        prev_digit = current_digit;
        temp /= 10;
    }

    if (is_inc) {
        printf("\nNumber: %d, Output: Digits are in increasing order\n", num);
    } else {
        printf("\nNumber: %d, Output: Digits are not in increasing order\n",
num);
    }
}

void checkDecreasing(void) {
```

```
    int num, temp, prev_digit = -1, is_dec = 1;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;
    while (temp > 0) {
        int current_digit = temp % 10;
        if (current_digit <= prev_digit) {
            is_dec = 0;
            break;
        }
        prev_digit = current_digit;
        temp /= 10;
    }

    if (is_dec) {
        printf("\nNumber: %d, Output: Digits are in decreasing order\n",
num);
    } else {
        printf("\nNumber: %d, Output: Digits are not in decreasing order\n",
num);
    }
}

void findLargestSmallest(void) {
    int num, temp, digit;
    int largest = -1, smallest = 10;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;
    while (temp > 0) {
        digit = temp % 10;
        if (digit > largest) largest = digit;
        if (digit < smallest) smallest = digit;
        temp /= 10;
    }

    printf("\nNumber: %d, Largest Digit = %d, Smallest Digit = %d\n", num,
largest, smallest);
```

```
}

void findSecondLargestSmallest(void) {
   int num, temp, digit;
   int freq[10] = {0};

   printf("Enter a positive integer: ");
   scanf("%d", &num);

   temp = num;
   while (temp > 0) {
      digit = temp % 10;
      freq[digit]++;
      temp /= 10;
   }

   int smallest = -1, second_smallest = -1;
   int largest = -1, second_largest = -1;

   for (int i = 0; i <= 9; i++) {
      if (freq[i] > 0) {
         if (smallest == -1) {
            smallest = i;
         } else if (second_smallest == -1) {
            second_smallest = i;
            break;
         }
      }
   }

   for (int i = 9; i >= 0; i--) {
      if (freq[i] > 0) {
         if (largest == -1) {
            largest = i;
         } else if (second_largest == -1) {
            second_largest = i;
            break;
         }
      }
   }
```

```
    if (second_largest != -1 && second_smallest != -1) {
        printf("\nNumber: %d, Second Largest = %d, Second Smallest =
%d\n", num, second_largest, second_smallest);
    } else {
        printf("\nNumber: %d, Output: Not enough distinct digits to find
second largest/smallest\n", num);
    }
}
```

6. Compilation, Execution & Screenshots